

Introduction to Algorithm

기초 알고리즘

Tractable vs Intractable 개념

Tractable vs. Intractable



지금까지의 관심사항

- 답을 '어떻게' 찾을 것인가?
 - 알고리즘: Divide-and Conquer, Greedy, Dynamic Programming, ...
- 답을 얼마나 '효율적으로' 찾을 것인가?
 - 시간복잡도: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, ...



Tractable vs. Intractable

관점의 전환: Can Solve → Can Solve in Time

- 답을 '유한한 시간내에' 찾을 수 있을까?
- 수학적으로 유한한 단계 내에 답을 구할 수 있다면 '풀 수 있는 문제 '
- 예를 들어, 1GHz CPU가 100년을 계산해야 답이 나온다면, 그것은 공학적으로 '풀 수 없는 문제 ' 와 같음
- '**현실적인 시간(Reasonable Time)**'이라는 잣대를 도입하여 문제를 재 분류



Tractable vs. Intractable

Tractable 문제

- 어원: 라틴어 tractare(다루다, 끌다)에서 유래
- 의미: 어떤 문제를 해결하기 위한 '효율적인 알고리즘(다항 시간)'이 존재하여,
컴퓨터가 현실적으로 통제하고 다룰 수 있는 문제
- 컴퓨터 과학적 정의: 시간 복잡도가 $O(n^k)$ (단, k 는 상수)로 표현되는 문제들



Intractable 문제

- 어원: 접두사 *in-*(부정) + *tractable*. 즉, 아무리 끌고 가려 해도 움직이지 않는 고집 센 상태
- 의미: 이론적으로는 답이 존재할 수 있지만, 문제를 풀기 위한 '지름길(효율적인 알고리즘)'이 없어서 **입력이 조금만 커져도 통제 불능 상태(시간 초과)가 되는 문제**
- 컴퓨터 과학적 정의: 다항 시간 내에 해결하는 알고리즘이 알려지지 않았으며, 시간복잡도가 $O(2^n)$ 또는 $O(n!)$ 로 표현되는 문제들



Tractable vs. Intractable 문제

- 입력 크기(n)가 늘어난다는 것은 컴퓨터가 처리해야 할 공간이 커짐을 의미
- 시간복잡도가 $O(n^k)$ 인 다항 시간 알고리즘은, 데이터가 2배 늘면 시간이 2배 혹은 4배로 선형적으로 증가 (각각 $k=1, k=2$ 인 예시이며, 일반적으로는 2^k 배 증가)
- 시간복잡도가 $O(2^n)$ 인 지수 시간 알고리즘은, 데이터가 1만 늘어나도 시간이 2배씩 '폭발적으로 증가'



Tractable vs. Intractable

Tractable vs. Intractable 예제

- 문제: 내가 가진 카드($N=500,000$) 중, 특정 카드($M=500,000$)가 있는지 확인
- 문제풀이 제한시간: **2초**



Tractable vs. Intractable 예제

- 풀이1: Intractable 한 접근 방식
 - 전략: M개의 카드 하나하나를 볼 때마다, 나의 카드 N개를 처음부터 끝까지 탐색
 - 시간 복잡도: $O(M \times N)$
 - 연산 횟수: $500,000 \times 500,000 = 250,000,000,000$ (**2,500억번**)
 - 컴퓨터가 1초에 1억번(10^8)연산한다고 가정하면, **약 2,500초(41분40초) 소요**



Tractable vs. Intractable 예제

- 풀이2: Tractable 한 접근 방식 → Binary Search
 - 전략: 카드 N개를 먼저 정렬($O(N \log N)$)한 뒤, M개의 카드 각각에 대해 이분탐색($O(M \log N)$)
 - 시간 복잡도: $O(N \log N + M \log N)$
 - 연산 횟수: $500,000 \times 19 + 500,000 \times 19 = \text{약 } 19,000,000$ (**1,900만번**)
 $250,000,000,000$ (2,500억번)
 - 컴퓨터가 1초에 1억번(10^8)연산한다고 가정하면, **0.19초 소요**

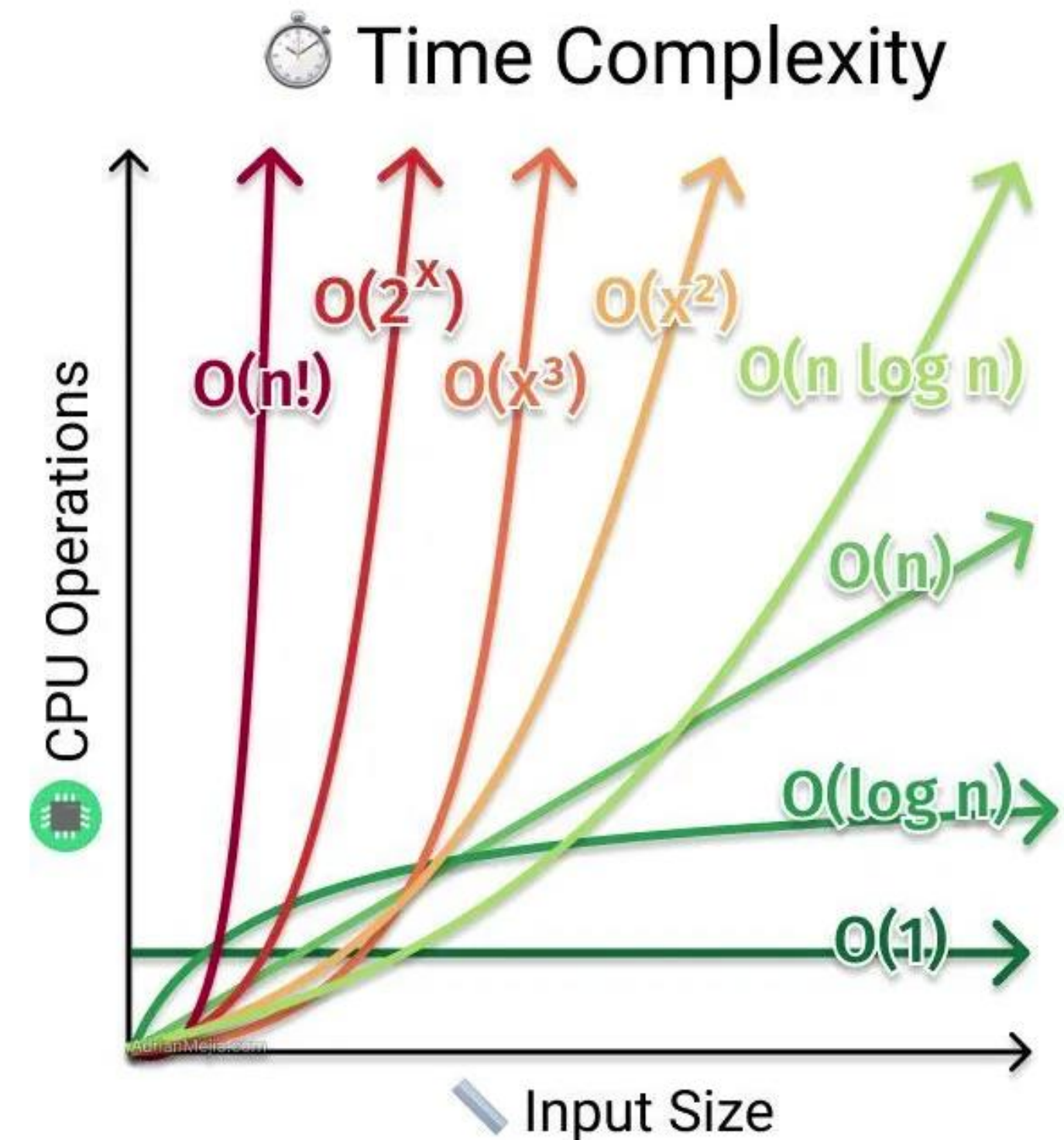


Tractable vs. Intractable

Intractable → Tractable 문제로 변환

- Tractable: $O(n^k)$
- Intractable: $O(2^n)$ 또는 $O(n!)$ 로 표현

풀이시간에 제약이 있고 N이 크다면,
Intractable 에서 Tractable 구조로
문제 재정의 필요!



결정 문제 vs. 최적화 문제



결정 문제 (Decision Problem)

- 예(Yes) 혹은 아니오(No)로 답할 수 있는 문제
- 복잡도 이론의 기초가 되며, 답의 '검증'이 명확
- 예) "이 그래프에서 가중치 합이 50 이하인 경로가 있는가?", "배낭에 10kg 이하로 담아 가치 100을 넘길 수 있는가?"



최적화 문제 (Optimization Problem)

- 가능한 여러 해 중에서 가장 좋은(최소값 혹은 최대값) 해를 찾는 문제.
- 예) "이 그래프에서 가장 짧은 경로는?", "배낭에 담을 수 있는 최대 가치는?"

→ 우리가 실생활에서 만나는 대부분의 문제들은 최적화 문제이다!

"우리집까지 가는 최단 경로는?", "트럭에 물건을 가장 많이 싣는 방법은?",
"수익을 극대화하는 매수/매도 시점은?", ...



결정 문제 vs. 최적화 문제

- 결정 문제는 답의 형태가 항상 "예/아니오" 이고, "조건 만족 여부" 라는 검증기준이 있음 → 문제 난이도 측정이 가능
 - 최적화 문제는 답의 형태가 다양(숫자, 경로, 집합 등)하고, "이게 정말 최소/최대인가?" 를 증명해야 함 → 문제 난이도를 측정하기 어려움
- ➔ '최적화 문제'를 '결정 문제' 로 변환 할 수 있다면, 서로 다른 분야의 문제의 난이도를 측정하고 비교 할 수 있을 것이다!**



최적화 문제 → 결정 문제로 변환

- 최적화 문제를 푸는 것은 결정 문제를 반복해서 푸는 것과 같다
- 즉, "가장 좋은 답(최적해)을 찾아라"라는 어려운 최적화 문제를, "답이 K보다 크다/작은가?" 라는 질문으로 단순화 시킬 수 있다
- 예) "배낭에 담을 수 있는 최대 가치는 얼마인가?" → "배낭에 가치가 K 이상이 되도록 담을 수 있는가? (Yes/No)"

➔ 결정문제를 푸는 것은 최적화 문제를 푸는 것보다 쉽거나 같다



"결정문제를 푸는 것은 최적화 문제를 푸는 것보다 쉽거나 같다"

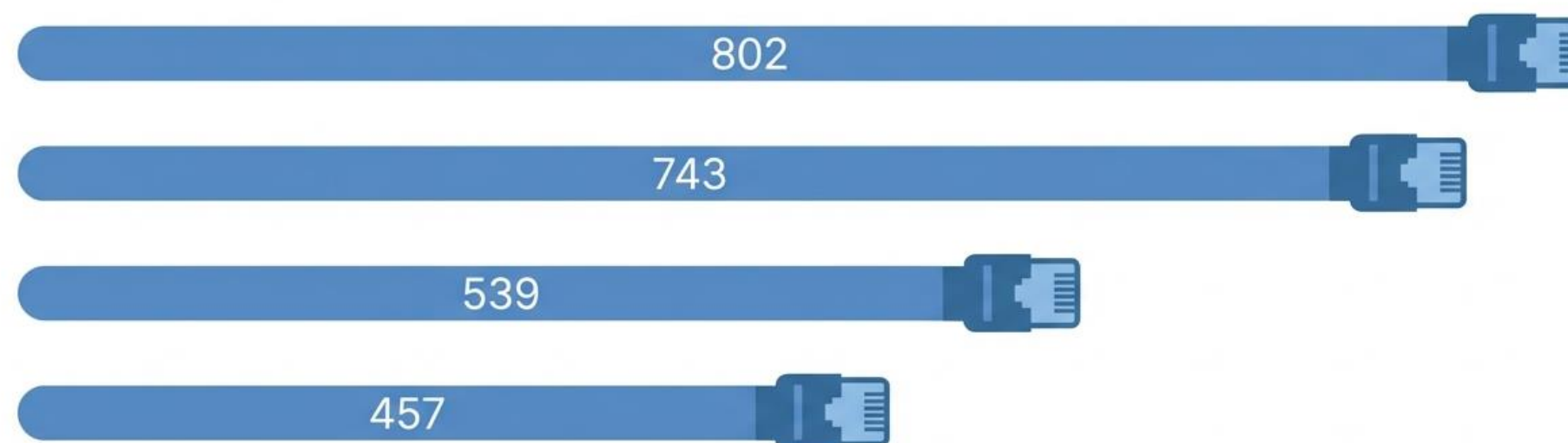
- "합이 K인 부분집합이 있는가?" 라는 결정 문제가 풀기 어렵다면, "합이 최대인 부분집합은?" 이라는 최적화 문제도 풀기 어려움
- "수익이 1억 원 이상인 투자 방법이 있는가?"라는 결정 문제를 푸는데 컴퓨터가 100년 걸려야 대답할 수 있다면, "수익을 최대화하는 방법은 무엇인가? " 라는 최적화 문제는 최소 100년 이상 걸리는 극악의 난이도
- 즉, 결정 문제가 Intractable하면 최적화 문제도 당연히 Intractable

최적화 문제 풀이 예제



문제

- 여러 개의 랜선을 가지고 있을 때, 이를 잘라서 동일한 길이의 랜선 N개를 만들려고 합니다
- 현재 가지고 있는 랜선의 길이: [802, 743, 457, 539]
- 필요한 랜선의 개수 N: 11
- 이때 만들 수 있는 랜선의 "최대 길이"를 구하시오





문제 정의

- 여러 개의 랜선을 가지고 있을 때, 이를 잘라서 동일한 길이의 랜선 N개를 만들려고 합니다
- 현재 가지고 있는 랜선들의 길이: [802, 743, 457, 539]
- 필요한 랜선의 개수 N: 11
- 이때 만들 수 있는 랜선의 "최대 길이"를 구하시오

➔ N개 이상의 랜선을 만들 수 있는 "최대 길이" 를 구하라 >> 최적화 문제



문제 정의

- 여러 개의 랜선을 가지고 있을 때, 이를 잘라서 동일한 길이의 랜선 N개를 만들려고 합니다
- 현재 가지고 있는 랜선들의 길이: [802, 743, 457, 539]
- 필요한 랜선의 개수 N: 11
- 이때 만들 수 있는 랜선의 "최대 길이"를 구하시오

➔ N개 이상의 랜선을 만들 수 있는 "최대 길이(X)" 를 구하라 >> 최적화 문제

➔ (변환) 길이가 X 일 때, N개 이상의 랜선을 만들 수 있는가? >> 결정 문제



문제 정의

- 길이가 X 일 때, N 개 이상의 랜선을 만들 수 있는가? >> 결정 문제
- Yes/No 판단 로직
 - 각 랜선의 길이를 X 로 나눈 몫을 모두 더한다 (== 랜선의 개수)
 - 합계가 N 이상이면 Yes, 아니면 No



문제 풀이 (결정문제 반복)

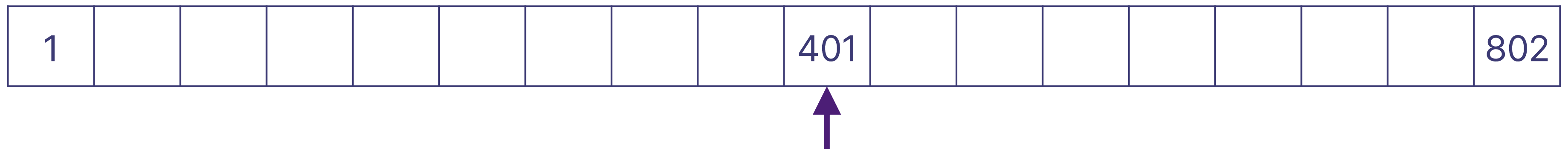
- 길이(X)를 바꿔가며 반복적으로 탐색하며, 길이가 X로 잘린 랜선의 개수가 N개 이상인지 확인(Yes/No 결정문제)
- 길이 탐색 방법
 - 1, 2, 3, ... 순차적으로 물어보는 것은 비효율적 ($O(\text{Max Length})$)
 - 파라메트릭 서치를 적용하여 중간값 부터 탐색
 - 확인 결과,
 - Yes 라면, 더 긴 길이도 가능한지 확인하기 위해 오른쪽 탐색
 - No 라면, 더 짧은 길이는 가능한지 확인하기 위해 왼쪽 탐색



문제 풀이 (결정문제 반복)

- 파라메트릭 서치를 적용하여 길이(X)를 바꿔가며 반복적으로 탐색
 - 현재 가지고 있는 랜선들의 길이: [802, 743, 457, 539]
 - 필요한 랜선의 개수 N: 11

➔ 탐색범위 (1~802)의 중간값(401) 부터 탐색 시작





문제 풀이 (결정문제 반복)

- 길이가 X로 잘린 랜선의 총 개수: 5
 - 802: $802 // 401 = 2$
 - 743: $743 // 401 = 1$
 - 457: $457 // 401 = 2$
 - 539: $539 // 401 = 1$

➔ 결정문제 결과: No (11개 미만)

1									401								802
---	--	--	--	--	--	--	--	--	-----	--	--	--	--	--	--	--	-----



문제 풀이 (결정문제 반복)

- 길이가 401인 경우, 결정문제 결과 No → 더 짧은 길이(좌측) 탐색
- 탐색범위 (1~400)의 중간값(200) 부터 탐색





문제 풀이 (결정문제 반복)

- 길이가 X로 잘린 랜선의 총 개수: 11
 - 802: $802 // 200 = 4$
 - 743: $743 // 200 = 3$
 - 457: $457 // 200 = 2$
 - 539: $539 // 200 = 2$

➔ 결정문제 결과: Yes (11개 이상)

1				200				400	401								802
---	--	--	--	-----	--	--	--	-----	-----	--	--	--	--	--	--	--	-----





문제 풀이 (결정문제 반복)

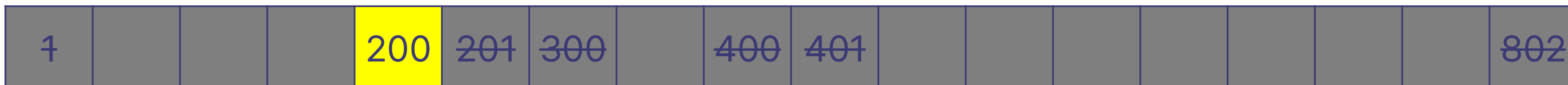
- 길이가 200인 경우, 결정문제 결과 Yes → 더 긴 길이(우측) 탐색
- 탐색범위 (201~400)의 중간값(300) 부터 탐색





문제 풀이 (결정문제 반복)

- 이 과정을 반복하면,
 - 탐색범위: $[1, 802] \rightarrow$ 길이(중간값): 401 $\rightarrow$ 결정문제: No
 - **탐색범위: $[1, 400] \rightarrow$ 길이(중간값): 200 $\rightarrow$ 결정문제: Yes (N: 11개)**
 - 탐색범위: $[201, 400] \rightarrow$ 길이(중간값): 300 $\rightarrow$ 결정문제: No
 - ...
 - 탐색범위: $[201, 201] \rightarrow$ 길이(중간값): 201 $\rightarrow$ 결정문제: No





문제 풀이 결과

- “최대 길이를 구하라” 라는 막연한 질문 대신, “길이(X)로 자르면 11개가 나오는가?” 라는 단순한 결정 문제로 변환
- 탐색과정에서, 이분 탐색을 통해 빠르게 최적값 탐색
 - 1부터 802 까지 순차 탐색했을 경우, 최대 802번 반복 탐색
 - 이분 탐색을 활용하면 10번 내외 ($\log_2 802 \approx 9.6$) 탐색
- 최종적으로 결정문제의 반복을 통해 최대길이(200)를 찾아냄

P vs. NP



P 클래스란?

- P: Polynomial time
- **다항 시간($O(n^k)$) 안에 해답을 찾을 수 있는 모든 결정 문제의 집합**
- 컴퓨터가 현실적으로 잘 푸는 Tractable 문제 영역이며, 우리가 '효율적'이라고 부르는 대부분의 알고리즘(정렬, 이분 탐색, MST 등)이 여기에 속함



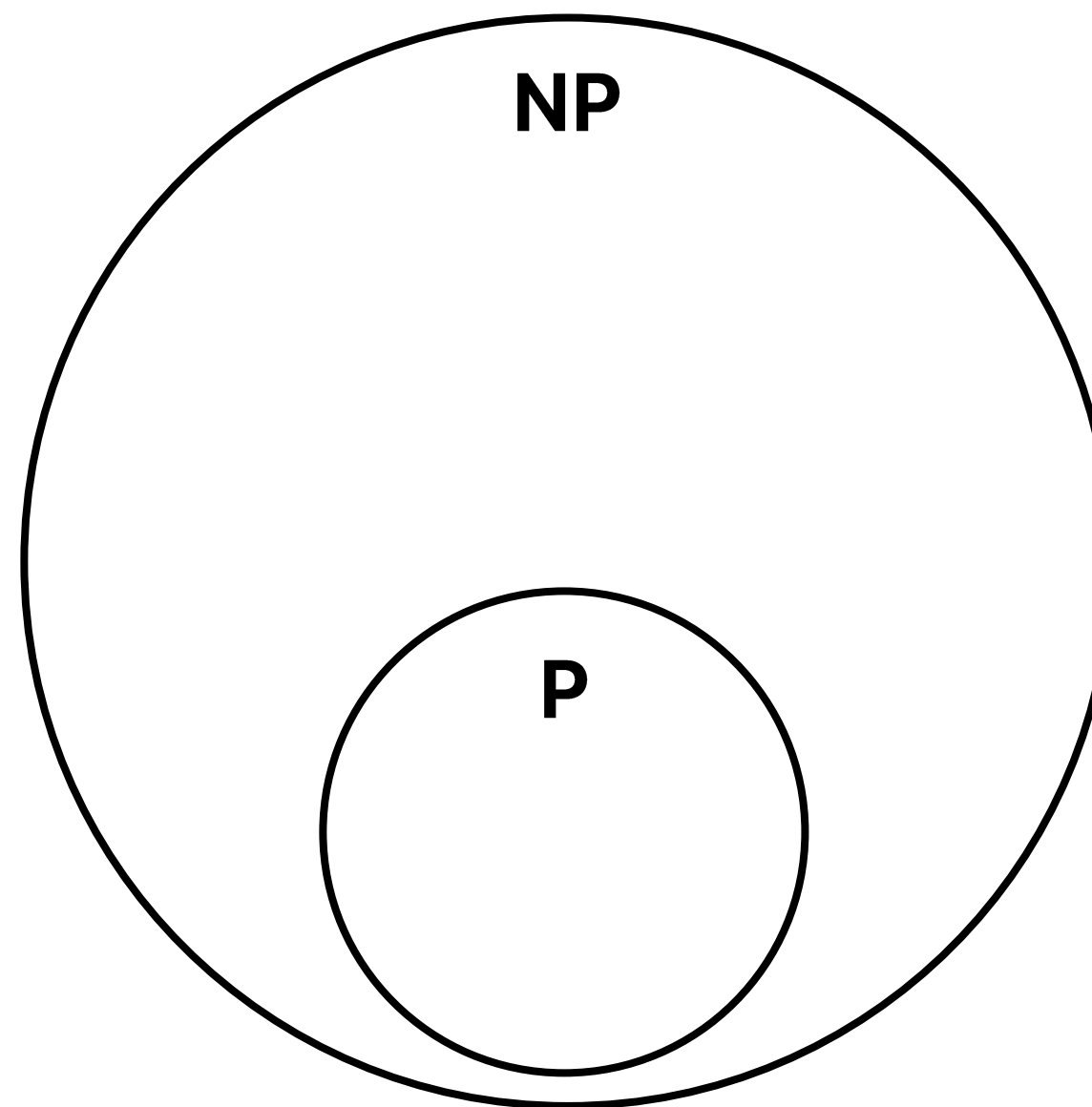
NP 클래스란?

- NP: Non-deterministic Polynomial time
- 직접 답을 찾기는 어려울지 몰라도, 누군가 "이게 답이야"라고 가져왔을 때 그것이 맞는지 **확인(Verification)**하는 것만은 **다항 시간 안에** 할 수 있는 문제들의 집합
- 예) "스도쿠나 퍼즐은 풀기는 매우 힘들지만(Intractable), 남이 풀어놓은 걸 보면 맞는지 확인하는 건 빠르게 할 수 있음(Tractable)"



P vs. NP

- 빠르게 풀 수 있는 문제(P)는, 확인도 빠르게 할 수 있다(NP)
- 즉, 모든 P 문제는 NP 문제에 포함된다





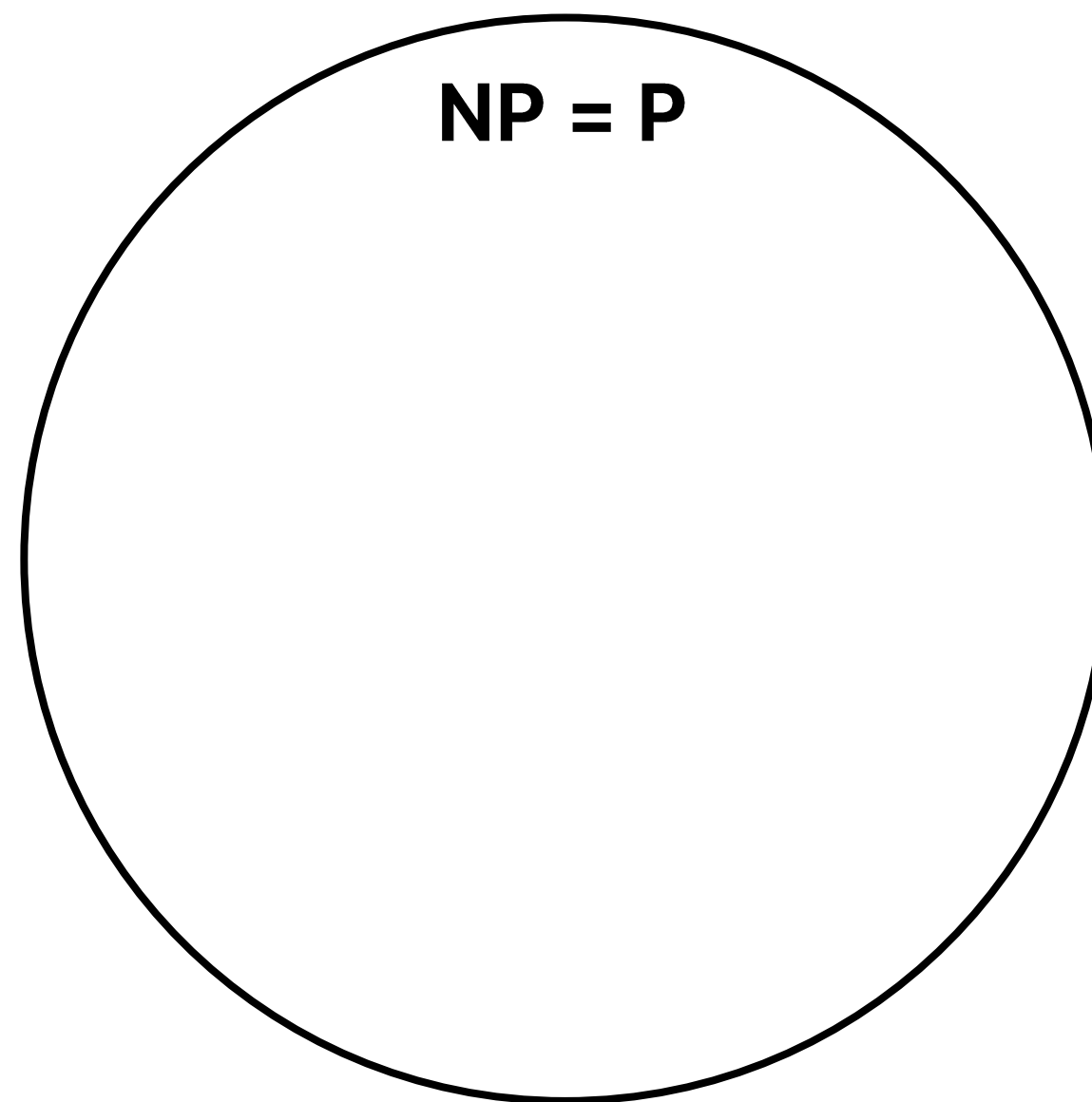
P vs. NP

- 빠르게 풀수 있는 문제(P)는, 확인도 빠르게 할 수 있다(NP)
 - 즉, 모든 P 문제는 NP 문제에 포함된다
- ➔ 의문: 답을 "확인" 하는 것이 빠르다면, 답을 "찾는" 것도 빠르게 할 수 있지 않을까?
 - 확인(NP)은 쉬운데 찾기(P)는 왜 이토록 어려운가? 혹시 우리가 아직 엄청나게 똑똑한 '찾기 방법'을 발견하지 못한 것뿐일까?



P vs. NP

- 만일, 모든 NP 문제가 P에 포함된다면?
 - 즉, $P = NP$ 라면?



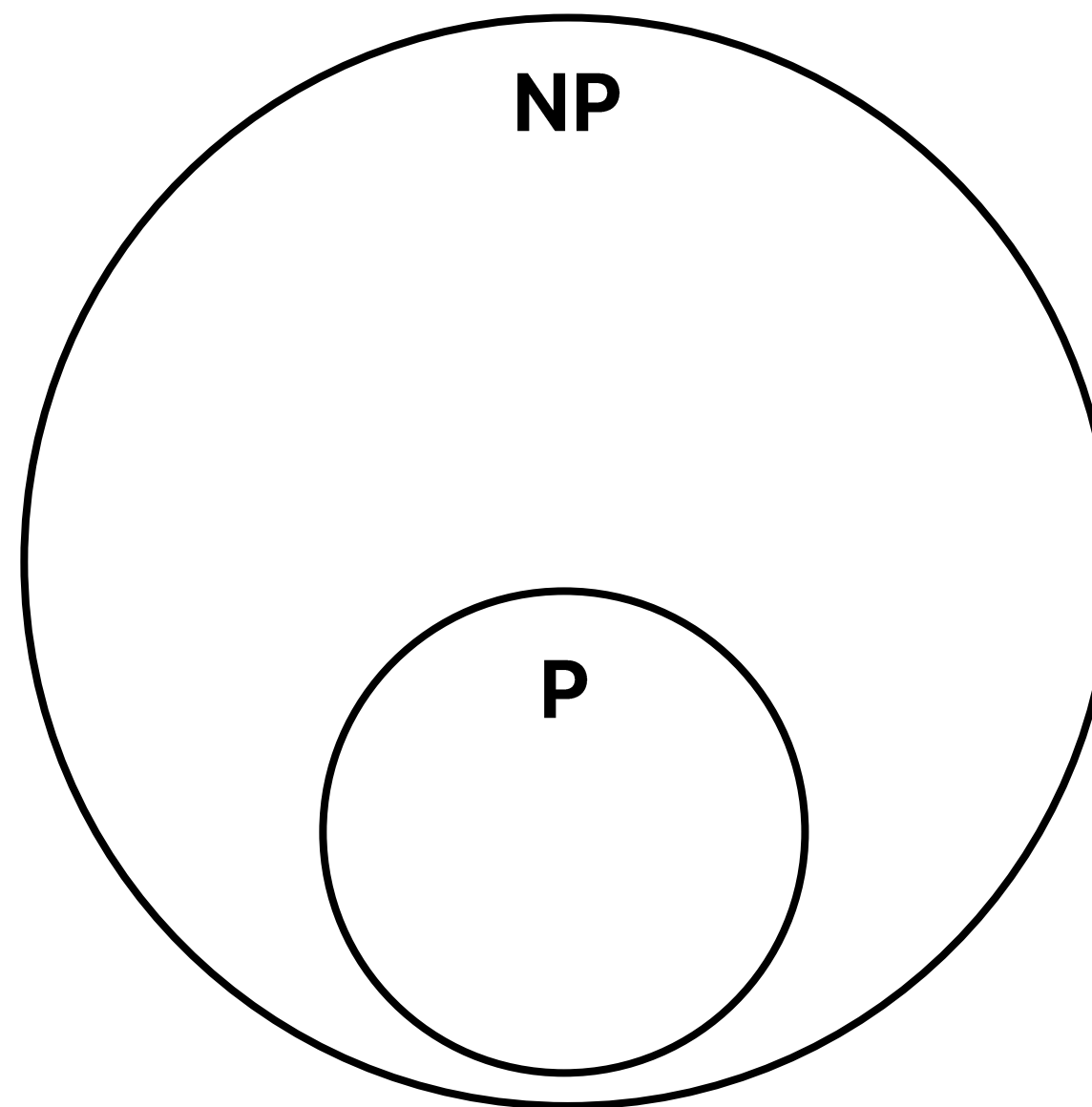


P vs. NP

- 만약, $P = NP$ 라면 (증명 할 수 있다면),
 - 답을 확인하는 것이 빠르면(NP) 답을 찾는 것도 빠르게(P) 할 수 있음
 - 인류의 수많은 난제(암호 해독, 암 치료 등)들을 해결 가능
 - 암 치료: 수만 개의 단백질 조합 중 치료에 적합한 구조를 찾는 일을 다항시간 안에 해결
 - 물류 최적화: 전 세계 모든 택배 경로를 가장 효율적으로 짜는 일을 초 단위로 처리

P vs. NP

- 만일, 모든 NP 문제가 P 에 포함되는건 아니라면?
- 즉, $P \neq NP$ 라면?





P vs. NP

- 만약, $P \neq NP$ 라면 (증명할 수 있다면),
 - 확인은 쉽지만(NP), 정답을 찾아내는 데는 지수 시간이 걸리는 문제가 존재
 - 현대 금융과 보안 체계는 “비밀번호를 확인하는 건 쉽지만, 직접 찾아내는 건 불가능에 가까울 정도로 오래 걸린다” 는 믿음에 기반
 - 대부분의 학자들은 $P \neq NP$ 라고 믿고 있지만, 수학적으로 “증명” 하는 것은 불가능에 가까운 난제로 남아 있음